

UNIVERSIDAD NACIONAL DEL ALTIPLANO

Puno

Escuela Profesional de Ingeniería de Sistemas

**Práctica 01 — Búsquedas y
Notación Asintótica**

Implementación y comparación experimental en Python y C++

Curso

Algoritmos y Estructuras de Datos – SIS210

Docente

ZANABRIA GALVEZ ALDO HERNAN

Integrantes

YANA MENDOZA CARLOS BENEDICTO

BELIZARIO YANA DAVID VICTOR

CURASI ZEVALLOS HANDDY RONALD

Puno – Perú

2026

1. Estado del arte y fundamento teórico

Buscar una clave en un arreglo exige distinguir las garantías del algoritmo de las características de la entrada. Lineal admite datos sin ordenar y revisa elementos sucesivos; binaria exige orden y descarta aproximadamente la mitad del intervalo por iteración. Los textos de Cormen et al. (2022) y Sedgewick y Wayne (2011) sitúan esta comparación dentro del análisis de crecimiento y del estudio experimental: contar operaciones y medir duración responden preguntas complementarias.

Exponencial obtiene una cota mediante índices $1, 2, 4, 8, \dots$ y aplica binaria al intervalo delimitado. Su costo depende de la posición alcanzada, $O(\log(p+1))$, y resulta útil cuando existe acceso indexado seguro pero no se conoce inicialmente el final. Interpolación estima

$$\text{pos} = \ell + \left\lfloor \frac{(x - a[\ell])(h - \ell)}{a[h] - a[\ell]} \right\rfloor,$$

con controles de rango y de extremos iguales. Perl et al. (1978) vinculan su costo esperado de accesos, $O(\log \log n)$, a claves uniformemente distribuidas; una distribución sesgada puede invalidar esa ventaja. Van Sandt et al. (2019) estudian versiones optimizadas que intercambian cálculo por menos accesos a memoria y adaptan la búsqueda a la distribución. Esto amplía el enfoque clásico: ni el nombre del algoritmo ni su orden asintótico bastan para anticipar su rendimiento práctico. Aquí se evalúan versiones manuales básicas, no esas variantes optimizadas.

Notación. $O(g(n))$ acota superiormente una función de costo hasta una constante, para n suficientemente grande; $\Omega(g(n))$ la acota inferiormente y $\Theta(g(n))$ reúne ambas cotas. No equivalen, respectivamente, a peor, mejor y promedio: los casos se definen por separado. Bajo un modelo RAM de costo unitario, las versiones iterativas usan $O(1)$ espacio auxiliar.

Algoritmo	Mejor	Promedio	Peor	Supuesto del promedio
Lineal	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$	Posiciones equiprobables; ausencia recorre todo.
Binaria	$\Theta(1)$	$\Theta(\log n)$	$\Theta(\log n)$	Orden, claves distintas y posiciones equiprobables.
Exponencial	$\Theta(1)$	$\Theta(\log n)$	$\Theta(\log n)$	Mismo modelo; claves próximas al inicio cuestan menos.
Interpolación	$\Theta(1)$	$O(\log \log n)$	$\Theta(n)$	Esperado bajo uniformidad probabilística; no universal.

2. Metodología y correctitud

Se siguió la guía oficial (Zanabria Galvez, s.f.) con $n \in \{10\,000, 100\,000, 500\,000\}$. Python generó los archivos compartidos con semilla 123: (a) uniforme consecutivo $0, \dots, n-1$, sin duplicados; (b) sesgado, exactamente 80% en $[0, n/10 - 1]$ y 20% en $[n/10, 5n]$, con duplicados posibles, luego ordenado; (c) una permutación del uniforme. Se guardaron huellas SHA-256 de los nueve archivos.

Cada archivo contiene 1000 consultas: 700 valores tomados por posición con reemplazo y 300 ausentes $10n + i$, $0 \leq i < 300$; se mezclaron con el mismo generador. Ambos lenguajes leyeron idénticos datos y consultas. Solo lineal se ejecutó sobre desordenados. Se validaron vacío, único, repetidos, negativos, extremos, intermedios, hits, misses y 50 arreglos aleatorios; además, cada consulta experimental se comprobó antes de medir. Se acepta cualquier índice que contenga la clave; -1 solo si está ausente.

Se midieron tres lotes de 1000 búsquedas por combinación (27 combinaciones por lenguaje), con 20 consultas de calentamiento y orden de algoritmos rotado. Se usaron `steady_clock` de `std::chrono` y `time.perf_counter()`; se dividió cada lote entre 1000 y se promediaron sus tiempos. Lectura, generación, ordenamiento y validación quedaron fuera; ciclo, llamada y checksum quedaron dentro. El conteo de comparaciones se ejecutó aparte en C++, sin modificar las funciones cronometradas. Se conservaron CSV de medias, desviaciones muestrales, repeticiones y registros de consola.

3. Entorno experimental y límites del diseño

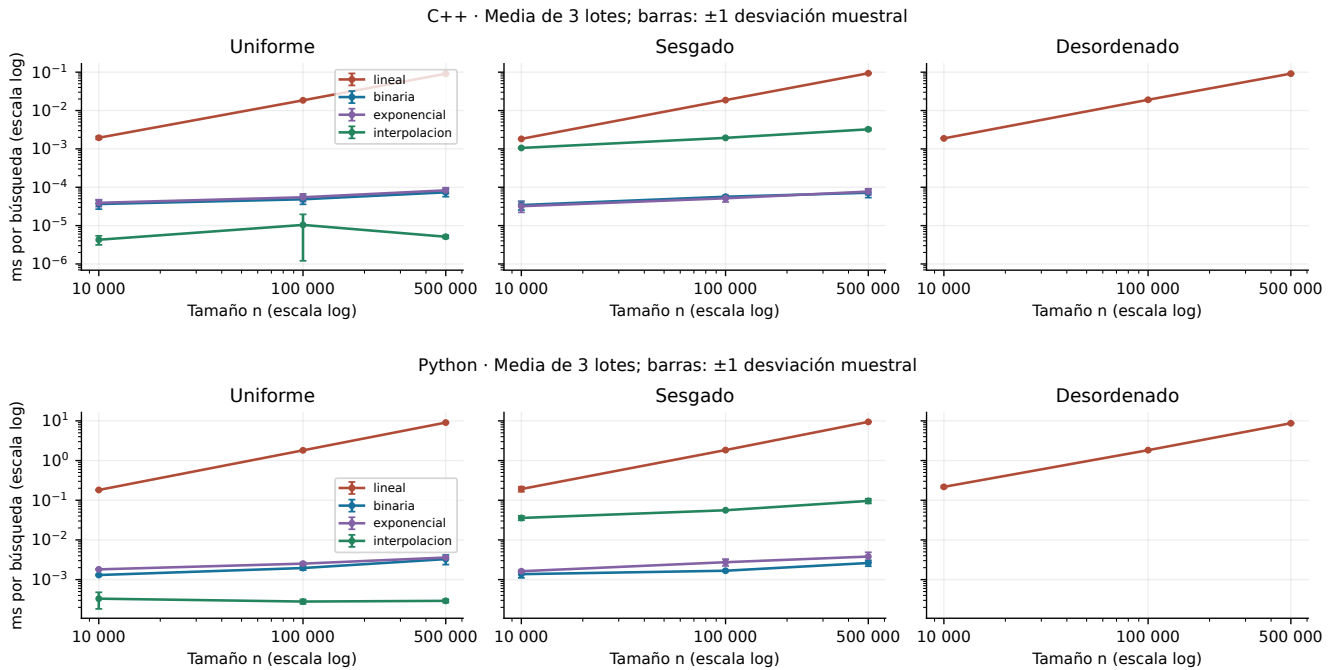
Entorno remoto de Work: Ubuntu 24.04.3 LTS, Linux-6.18.35-x86_64-with-glibc2.39; CPU visible AMD EPYC 9V74 80-Core Processor; RAM visible 16399700 KiB (aprox. 15.64 GiB), límite del contenedor 14 GiB. Python 3.12.14; g++ (Ubuntu 13.3.0-6ubuntu2 24.04) 13.3.0. Flags: `-std=c++17 -O2 -Wall -Wextra -pedantic`. Ejecución secuencial, sin fijar afinidad ni aislamiento exclusivo; el nombre del procesador no representa núcleos dedicados. Registro UTC: 2026-09-09.

Los misses externos favorecen el rechazo inmediato de interpolación. El uniforme consecutivo permite estimación exacta y no representa todos los datos aleatorios uniformes. Las pruebas previas calientan memoria; no se mide caché fría. Tres repeticiones, lotes C++ muy cortos y una sola semilla limitan inferencias: se describen tendencias, no significancia estadística ni pruebas matemáticas de complejidad.

4. Resultados experimentales

Tabla 1. Media en milisegundos por búsqueda; tres lotes, 700 hits y 300 misses por lote. Se muestran todas las combinaciones válidas; «-» indica precondition incumplida. Notación e-05 significa $\times 10^{-5}$.

Lenguaje	Dataset	n	Lineal	Binaria	Exponencial	Interpolación
C++	uniforme	10 000	0.00195063	3.65003e-05	3.92607e-05	4.27967e-06
C++	uniforme	100 000	0.018463	4.8508e-05	5.4807e-05	1.04223e-05
C++	uniforme	500 000	0.091155	7.3752e-05	8.29853e-05	5.12767e-06
C++	sesgado	10 000	0.0018231	3.43477e-05	3.20403e-05	0.00105852
C++	sesgado	100 000	0.0186869	5.6603e-05	5.15223e-05	0.00193876
C++	sesgado	500 000	0.0936415	7.18157e-05	7.71733e-05	0.00323955
C++	desordenado	10 000	0.00187829	-	-	-
C++	desordenado	100 000	0.0190711	-	-	-
C++	desordenado	500 000	0.0917346	-	-	-
Python	uniforme	10 000	0.181515	0.00130485	0.00181138	0.00033091
Python	uniforme	100 000	1.80999	0.00195744	0.00252695	0.000280049
Python	uniforme	500 000	9.05581	0.00328532	0.00360516	0.000291589
Python	sesgado	10 000	0.191669	0.00137221	0.00162627	0.0357729
Python	sesgado	100 000	1.83819	0.00167438	0.00274535	0.059274
Python	sesgado	500 000	9.44253	0.00262187	0.00380804	0.0962232
Python	desordenado	10 000	0.217395	-	-	-
Python	desordenado	100 000	1.82729	-	-	-
Python	desordenado	500 000	8.72613	-	-	-



Figuras 1 y 2. Curvas generadas directamente desde los CSV; ambos ejes son logarítmicos. Las barras representan una desviación muestral entre lotes, no intervalos de confianza. Cada lenguaje tiene su propia escala vertical.

Al multiplicar n por 50 en el uniforme, el tiempo lineal aumentó 46.7 veces en C++ y 49.9 en Python: tendencia compatible con crecimiento lineal, con constantes y ruido reales. Para $n = 500\,000$, binaria fue 1236.0 y 2756.4 veces más rápida que lineal, respectivamente. Interpolación fue 14.4 y 11.3 veces más rápida que binaria en el uniforme, pero tardó 45.1 y 36.7 veces lo de binaria en el sesgado.

Conteo independiente. En C++, para el uniforme, lineal pasó de 6542.549 a 326242.621 comparaciones; binaria de 24.878 a 35.934 e interpolación permaneció en 3.4. En el sesgado de 500 000, binaria registró 32.4 e interpolación 1600.14. Se cuentan comparaciones entre valores, incluidos límites y extremos; no controles de índices ni aritmética. El gráfico completo está en [graficos/comparaciones.pdf](#). Los conteos respaldan la interpretación sin depender de milisegundos.

5. Discusión: respuestas a las preguntas de la práctica

1. **¿Por qué binaria supera a lineal en datos ordenados?** El orden permite descartar mitades. En el uniforme grande se obtuvieron unas 36 comparaciones frente a 326 mil de lineal. La menor cantidad de pasos compensa aquí el acceso menos secuencial de binaria.
2. **¿Cuándo interpolación puede superar a binaria?** Cuando valor y posición se relacionan aproximadamente de manera lineal. En nuestros enteros consecutivos la estimación es exacta; cada hit requiere una sola posición estimada. La media de 3.4 comparaciones incluye cuatro por hit y dos por miss externo.
3. **¿Cuándo pierde la ventaja?** Ante concentraciones, huecos grandes o valores extremos que distorsionan la estimación. El sesgado lo evidencia, sin llegar a demostrar el peor caso. Un arreglo $[0, 1, \dots, n-2, n^2]$ buscando $n-2$ puede provocar avances casi unitarios y costo lineal.
4. **¿Por qué afecta el sesgo?** El 80% ocupa solo un rango numérico estrecho: la fracción de valores ya no representa la fracción de posiciones. La fórmula subestima posiciones en esa zona y necesita reiterar; binaria depende del orden, no de la uniformidad numérica.
5. **¿Para qué sirve exponencial con límite desconocido?** Duplica índices hasta superar la clave o detectar fin y luego refina con binaria. Requiere acceso indexado seguro; un flujo únicamente secuencial no permite saltos gratuitos. Aquí conocemos n : se estudia la acotación, sin simular una fuente infinita ni medir esa ventaja específica.
6. **¿Cómo influyen caché, localidad, ramas y constantes?** Lineal lee posiciones consecutivas, favorables a localidad y precarga; binaria salta y decide ramas dependientes de datos. Interpolación añade multiplicación y división. C++ usa enteros contiguos; Python maneja objetos y ejecuta bytecode. Son mecanismos plausibles (Van Sandt et al., 2019), no causas aisladas: no se midieron fallos de caché ni de predicción de ramas.
7. **¿Por qué Big O no predice milisegundos?** Omite constantes y términos menores; tampoco fija hardware, representación, distribución de consultas o carga del sistema. Exponencial y binaria comparten orden logarítmico y aun así difieren. Las pequeñas ventajas de exponencial en algunos conjuntos C++ no justifican un ganador universal, especialmente con la dispersión observada.

6. Heurística propuesta

Se propone un selector sencillo, implementado y probado aparte en `python/heuristica.py`. Sea q el número previsto de consultas y E el error máximo entre posición relativa y valor normalizado, estimado con 33 posiciones equiespaciadas:

Si límite desconocido: exponencial con acceso seguro si hay orden certificado; si no, lineal hasta fin.

Si $n < 64$: lineal. Si no hay certificado de orden, verificarlo una vez en $O(n)$.

Si desordenado: comparar qn con $n \log_2 n + q \log_2 n$; ordenar una copia y usar binaria solo si se amortiza; en otro caso, lineal.

Si extremos iguales: binaria. En otro caso, muestrear y calcular $E = \max_i \left| \frac{a[i] - a[0]}{a[n-1] - a[0]} - \frac{i}{n-1} \right|$.

Si $E \leq 0.05$: interpolación; si no, binaria.

Los umbrales 64 y 0.05 son propuestos, no calibrados. El muestreo no certifica orden ni garantiza velocidad. El costo de verificar, copiar y ordenar debe amortizarse; para una sola consulta sin metadatos puede convenir lineal directamente. El modelo supone constantes iguales y orienta una decisión, no demuestra optimalidad. Si se requieren índices originales, se preservan al ordenar pares.

7. Conclusiones

Binaria ofrece una opción estable para datos ordenados no uniformes; interpolación destacó en el caso consecutivo y perdió claramente con sesgo. Lineal evita preparación en desordenados y pocas consultas; exponencial aporta acotación cuando la interfaz admite saltos sin conocer el final. Los resultados son reproducibles desde archivos compartidos y coherentes con estas explicaciones, pero no demuestran cotas asintóticas ni se extrapolan a cualquier equipo o distribución.

Referencias

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4th ed.). MIT Press. <https://mitpress.mit.edu/9780262046305/introduction-to-algorithms/>
- Perl, Y., Itai, A., & Avni, H. (1978). Interpolation search—a log log N search. *Communications of the ACM*, 21(7), 550–553. <https://doi.org/10.1145/359545.359557>
- Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed.). Addison-Wesley. <https://algs4.cs.princeton.edu/>
- Van Sandt, P., Chronis, Y., & Patel, J. M. (2019). Efficiently searching in-memory sorted arrays: Revenge of the interpolation search? In *Proceedings of the 2019 International Conference on Management of Data* (pp. 36–53). Association for Computing Machinery. <https://doi.org/10.1145/3299869.3300075>
- Zanabria Galvez, A. H. (s.f.). *Práctica 01 — Búsquedas y Notación Asintótica* [Guía de práctica, SIS210]. Universidad Nacional del Altiplano.